

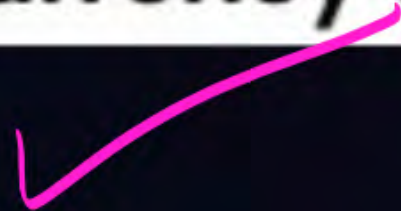
Computer Science & IT

Database Management System



Transactions & concurrency control

Lecture No. 11



By- Vishal Sir



Recap of Previous Lecture

- ✓ Topic Time stamp
- ✓ Topic Read time stamp
- ✓ Topic Write time stamp
- ✗ Topic Basic time stamp ordering protocol



Topics to be Covered



Topic

Time stamp ordering protocol with Thomas write rule

Topic

Wait-die protocol

Topic

Wound-wait protocol





Topic : Time stamp ordering protocols

There are two different versions of time stamp ordering Protocol

- ① Basic time stamp ordering protocol
- ② Time stamp ordering protocol with Thomas Write Rule

Time Stamp :-

Time stamp is a unique value assigned by DBMS to each transaction in ascending order.

• Let T_1 & T_2 are two transactions such that

$$TS(T_1) < TS(T_2)$$

Time Stamp
of transaction T_1

then T_1 is old transaction
& T_2 is young transaction

Time Stamp of
transaction T_2

eg: RTS(A) & WTS(A) :-

	$TS(T_1)=10$ T_1	$TS(T_2)=20$ T_2	$TS(T_3)=30$ T_3	$TS(T_4)=40$ T_4	RTS(A)	WTS(A)
initially					○	○
values after R(A) by T_1 →	R(A)				10	○
			R(A)		30	○
		W(A)			30	20
		R(A)			30	20
				W(A)	30	40
			W(A)		30	40



Topic : Basic Time stamp ordering protocol

(B.T.S.O.P.)

A schedule is allowed to execute using B.T.S.O.P. if and only if schedule is a Conflict serializable schedule and Conflict equivalent serial schedule is based on time stamp ordering of the transaction

eg Consider the schedule 'S' with time stamp ordering as specified along with transactions

T_1	T_2	T_3
$TS(T_1)=20$	$TS(T_2)=10$	$TS(T_3)=30$

We can observe that the time stamp ordering is

$$TS(T_2) < TS(T_1) < TS(T_3)$$

$$\text{i.e., } T_2 \rightarrow T_1 \rightarrow T_3$$

Schedule 'S' will be allowed to execute using B.T.S.O.P. if and only if "S" is a Conflict serializable schedule and 'S' is Conflict equivalent to serial schedule i.e. Serial schedule based on time stamp ordering
 $T_2 \rightarrow T_1 \rightarrow T_3$

Basic time stamp ordering protocol Conditions:-

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

① When transaction T_1 issue a Read(A) opⁿ



If T_1 is allowed to perform this R(A) opⁿ then also the behaviour of the schedule will be Conflict equivalent to serial schedule based on time stamp ordering of the transactions. {i.e. $T_1 \rightarrow T_2$ }

• T_1 is allowed to perform this R(A) opⁿ.

→ Reader will not have problem with other readers.



If T_1 is allowed to perform this R(A) opⁿ then behaviour of schedule will not be Conflict equivalent to the serial schedule based on time stamp ordering of the transaction {i.e. $T_1 \rightarrow T_2$ }

• Transaction T_1 is not allowed to perform this Read(A) opⁿ, and Rollback transaction T_1

* if transaction T_1 issue a Read(A) opⁿ and $W.T.S(A) > TS(T_1)$, then Rollback transaction T_1

Basic time stamp ordering protocol Conditions:-

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

② When transaction T_1 issue a Write(A) opⁿ

(i)

T_1	T_2
$W(A)$	$R(A)$

$TS(T_1) = 10$
 $TS(T_2) = 20$
∴ Time stamp
Ordering is
 $T_1 \rightarrow T_2$

Rollback
 T_1

If T_1 is allowed to perform this $W(A)$ opⁿ then behaviour of schedule will not be Conflict equivalent to the serial schedule based on time stamp ordering of the transaction i.e, $T_1 \rightarrow T_2$.

- ∴ Transaction T_1 is not allowed to perform this Write(A) opⁿ, and Rollback transaction T_1
- * if transaction T_1 issue a Write(A) opⁿ And $R.T.S(A) > TS(T_1)$, then Rollback transaction T_1

(ii)

T_1	T_2
$W(A)$	$W(A)$

$TS(T_1) = 10$
 $TS(T_2) = 20$
∴ Time stamp
Ordering is
 $T_1 \rightarrow T_2$

Rollback
 T_1

→ If T_1 is allowed to perform this $W(A)$ opⁿ then behaviour of schedule will not be Conflict equivalent to the serial schedule based on time stamp ordering of the transaction i.e, $T_1 \rightarrow T_2$.

- ∴ Transaction T_1 is not allowed to perform this Write(A) opⁿ, and Rollback transaction T_1
- * if transaction T_1 issue a Write(A) opⁿ And $W.T.S(A) > TS(T_1)$, then Rollback transaction T_1

Basic time stamp ordering Protocol Condition:-

Let T_1 & T_2 are two transactions s.t. $TS(T_1) < TS(T_2)$

ie, T_1 is older than T_2

① Let T_1 issue Read(A) opⁿ:-

(a) If $WTS(A) > TS(T_1)$, then Rollback T_1

(b) Otherwise, T_1 is allowed to perform the Read(A) opⁿ
∴ T_1 will perform Read(A) opⁿ,

And set $RTS(A) = \text{Max}(RTS(A), TS(T_1))$

② Let T_1 issue Write(A) opⁿ:-

(a) If $RTS(A) > TS(T_1)$, then Rollback T_1 .

(or) (b) If $WTS(A) > TS(T_1)$, then Rollback T_1 .

(or) (c) Otherwise, T_1 is allowed to perform Write(A) opⁿ.

∴ T_1 will perform the write(A) opⁿ

And set $WTS(A) = TS(T_1)$

Q:- Consider the following schedule

³⁰ T ₁	¹⁰ T ₂	²⁰ T ₃
✓ R(A)		
	✓ R(B)	
✓ W(C)		
	W(B)	
		W(A)

Which of the following time stamp ordering of the transaction will allow the schedule to be executed by B.T.S.O.P.

- (a) $TS.(T_1, T_2, T_3) = (30, 10, 20)$
- (b) $TS.(T_1, T_2, T_3) = (30, 20, 10)$
- (c) $TS.(T_1, T_2, T_3) = (10, 20, 30)$
- (d) $TS.(T_1, T_2, T_3) = (10, 30, 20)$

R(B) ✓
 R(C) • WTS(C) > TS(T₃)
 ∴ Roll back T₃

Q:- Consider the following schedule

³⁰ T ₁	²⁰ T ₂	¹⁰ T ₃
✓ R(A)		
	✓ R(B)	
✓ W(C)		
		✓ R(B)
		✓ R(C)
	W(B)	
		W(A)

Which of the following time stamp ordering of the transaction will allow the schedule to be executed by B.T.S.O.P.

- (a) $TS(T_1, T_2, T_3) = (30, 10, 20)$
- (b) $TS(T_1, T_2, T_3) = (30, 20, 10)$
- (c) $TS(T_1, T_2, T_3) = (10, 20, 30)$
- (d) $TS(T_1, T_2, T_3) = (10, 30, 20)$

WTS(C) > TS(T₃)
∴ Rollback T₃

Q:- Consider the following schedule

	¹⁰ T ₁	²⁰ T ₂	³⁰ T ₃
✓ R(A)			
		✓ R(B)	
✓ W(C)			
			✓ R(B)
		✓ R(C)	
	W(B)		
		W(A)	

Which of the following time stamp ordering of the transaction will allow the schedule to be executed by B.T.S.O.P.

- (a) $TS(T_1, T_2, T_3) = (30, 10, 20)$
- (b) $TS(T_1, T_2, T_3) = (30, 20, 10)$
- (c) $TS(T_1, T_2, T_3) = (10, 20, 30)$
- (d) $TS(T_1, T_2, T_3) = (10, 30, 20)$

✓
 $WTS(C) < TS(T_3)$
 $\therefore$ allowed

$RTS(B) > TS(T_2)$
 $\therefore$ Rollback T_2

Q:- Consider the following schedule

¹⁰ T ₁	³⁰ T ₂	²⁰ T ₃
✓ R(A)		
	✓ R(B)	
✓ W(C)		
		✓ R(B)
		✓ R(C)
	✓ W(B)	
		✓ W(A)

Which of the following time stamp ordering of the transaction will allow the schedule to be executed by B.T.S.O.P.

(a) $TS(T_1, T_2, T_3) = (30, 10, 20)$

(b) $TS(T_1, T_2, T_3) = (30, 20, 10)$

(c) $TS(T_1, T_2, T_3) = (10, 20, 30)$

(d) $TS(T_1, T_2, T_3) = (10, 30, 20)$

✓ R(C) $WTS(C) < TS(T_3)$
∴ allowed

✓ W(B) $RTS(B) < TS(T_2)$ No writer
∴ Allowed at B

✓ W(A) $RTS(A) < TS(T_3)$ No writer
∴ Allowed at A

Q:- Consider the following schedule

T_1	T_2	T_3
R(A)		(A)
	R(B)	(B)
W(C)		C T _c
		R _B
		R _A
	W(B)	V



Acyclic precedence graph
Topological order is
 $T_1 \rightarrow T_3 \rightarrow T_2$ (a)

R(B) Schedule is
Conflict equivalent
to serial schedule

$R(c)$ to serial schedule
 $T_1 \rightarrow T_3 \rightarrow T_2$,
 for it to be
 allowed by
 B.T.S.O.P.

W(A) $TS(T_1) < TS(T_3) < TS(T_2)$, the only option that satisfy the Cond'n is option (D)

Which of the following time stamp ordering of the transaction will allow the schedule to be executed by B.T.S.O.P.

Q) T.S. $(T_1, T_2, T_3) = (30, 10, 20)$

⑥ T.S. $(T_1, T_2, T_3) = (30, 20, 10)$

© T.S. $(T_1, T_2, T_3) = (10, 20, 30)$

d. $TS(T_1, T_2, T_3) = (10, 30, 20)$

- Note :-
- ① A schedule S is allowed to execute using B.T.S.O.P. if and only if schedule S is a Conflict serializable schedule and Conflict equivalent serial schedule is according to the Time stamp ordering of the transactions.
 - ② If schedule is not a Conflict serializable schedule then it will never be allowed to execute using B.T.S.O.P.
 - ③ If schedule is a C.S.S. but Conflict equivalent serial schedule is not as per time stamp ordering of the transactions then schedule is not allowed by B.T.S.O.P.

* B.T.S.O.P covers only conflict serializable schedules, but it does not cover the schedules that are view serializable but not conflict serializable,

∴ We define time stamp ordering protocol with "Thomas write rule"



Topic : Time stamp ordering protocol with Thomas write rule

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

Case ① When transaction T_1 issue a Read(A) opⁿ

(i)

T_1	T_2
R(A)	R(A)

(ii)

T_1	T_2
R(A)	W(A)

These three Case will be exactly same as B.T.S.O.P.

Case ② When transaction T_1 issue a Write (A) opⁿ

(i)

T_1	T_2
W(A)	R(A)

(ii)

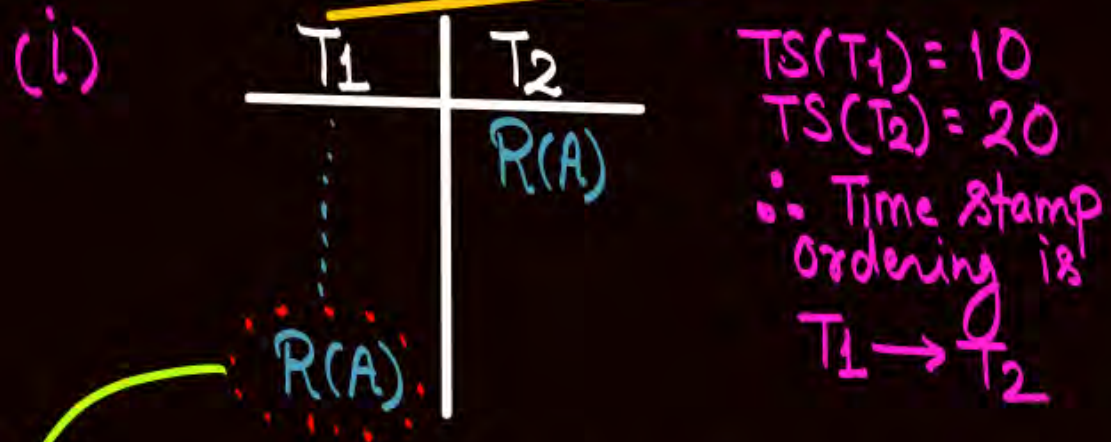
T_1	T_2
W(A)	W(A)

This case will be different from B.T.S.O.P.

Time stamp ordering protocol with Thomas write Rule: -

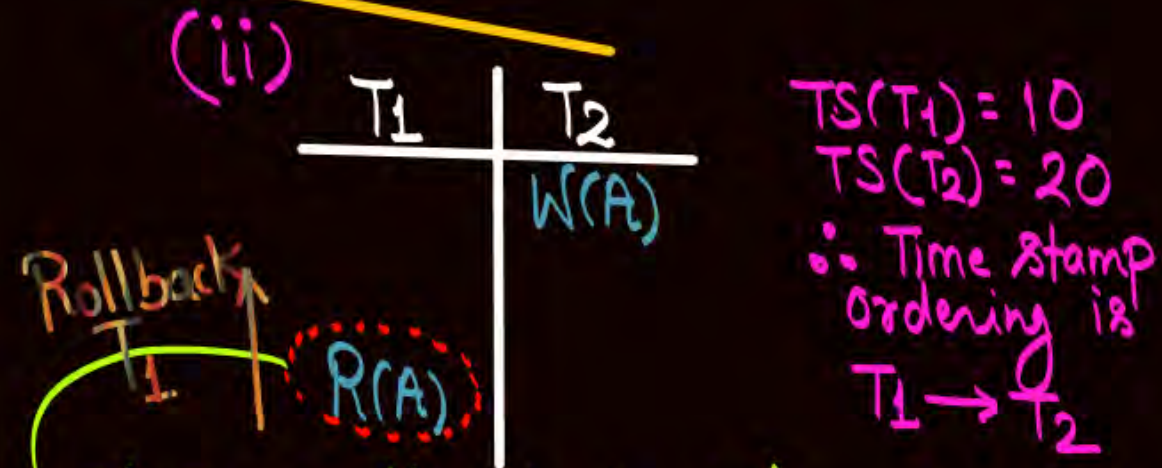
Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

① When transaction T_1 issue a Read(A) opⁿ



If T_1 is allowed to perform this R(A) opⁿ then also the behaviour of the schedule will be **view** equivalent to serial schedule based on time stamp ordering of the transactions. {i.e. $T_1 \rightarrow T_2$ }

- T_1 is allowed to perform this R(A) opⁿ.
- Reader will not have problem with other readers.



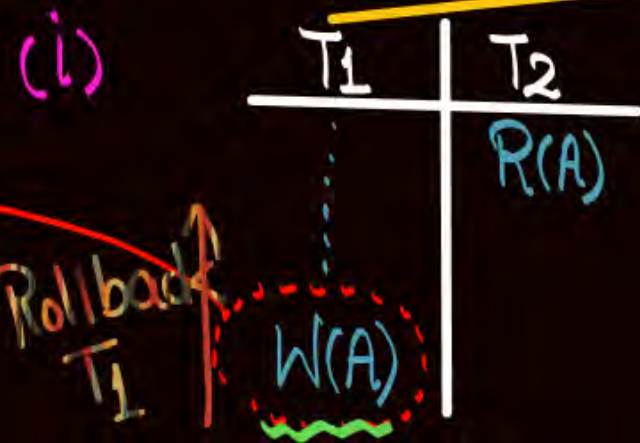
If T_1 is allowed to perform this R(A) opⁿ then behaviour of schedule will not be **view** equivalent to the serial schedule based on time stamp ordering of the transaction {i.e. $T_1 \rightarrow T_2$ }

- Transaction T_1 is not allowed to perform this Read(A) opⁿ, and Rollback transaction T_1
- if transaction T_1 issue a Read(A) opⁿ and $W.T.S(A) > TS(T_1)$, then Rollback transaction T_1

Time stamp ordering protocol with Thomas write Rule:-

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

② When transaction T_1 issue a Write(A) opⁿ



$TS(T_1) = 10$
 $TS(T_2) = 20$
∴ Time Stamp Ordering is
 $T_1 \rightarrow T_2$

If T_1 is allowed to perform this W(A) opⁿ then behaviour of schedule will not be view equivalent to the serial schedule based on time stamp ordering of the transaction {i.e., $T_1 \rightarrow T_2$ }

- ∴ Transaction T_1 is not allowed to perform this Write(A) opⁿ, and Rollback transaction T_1
- * if transaction T_1 issue a Write(A) opⁿ and $R.T.S(A) > TS(T_1)$, then Rollback transaction T_1

Time stamp ordering protocol with Thomas write Rule:-

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

(2) When transaction T_1 issue a Write(A) opn

(i)

T_1	T_2
$W(A)$	$R(A)$

$TS(T_1) = 10$
 $TS(T_2) = 20$
 ∴ Time stamp ordering is $T_1 \rightarrow T_2$

Rollback T_1

If T_1 is allowed to perform this $W(A)$ opn then behaviour of schedule will not be Conflict equivalent to the serial schedule based on time stamp ordering of the transaction i.e, $T_1 \rightarrow T_2$

- ∴ Transaction T_1 is not allowed to perform this Write(A) opn, and Rollback transaction T_1
- if transaction T_1 issue a Write(A) opn and $R.T.S(A) > TS(T_1)$, then Rollback transaction T_1

(ii)

T_1	T_2
$W(A)$	$W(A)$

$TS(T_1) = 10$
 $TS(T_2) = 20$
 ∴ Time stamp ordering is $T_1 \rightarrow T_2$

If transaction T_1 is allowed to perform this $W(A)$ opn then behaviour of schedule will not be view equivalent to serial schedule based on time stamp ordering

But if we skip this $W(A)$ opn of transaction T_1 , then schedule produced by Time stamp ordering protocol with Thomas write rule will be view equivalent to the serial schedule based on time stamp ordering of the transactions i.e if $WTS(A) > TS(T_1)$, then we will ignore that $W(A)$ of transaction T_1 and we will proceed with remaining operations of transaction T_1

Serial schedule $T_1 \rightarrow T_2$

T_1	T_2
-------	-------

$W(A)$

if we ignore the $W(A)$ opn of transaction T_1 , then schedule produced will be

T_1	T_2
	$W(A)$

Time Stamp Ordering Protocol with Thomas Write Rule:-

Let T_1 & T_2 are two transactions st. $TS(T_1) < TS(T_2)$

ie, T_1 is older than T_2

① Let T_1 issue Read(A) opⁿ:-

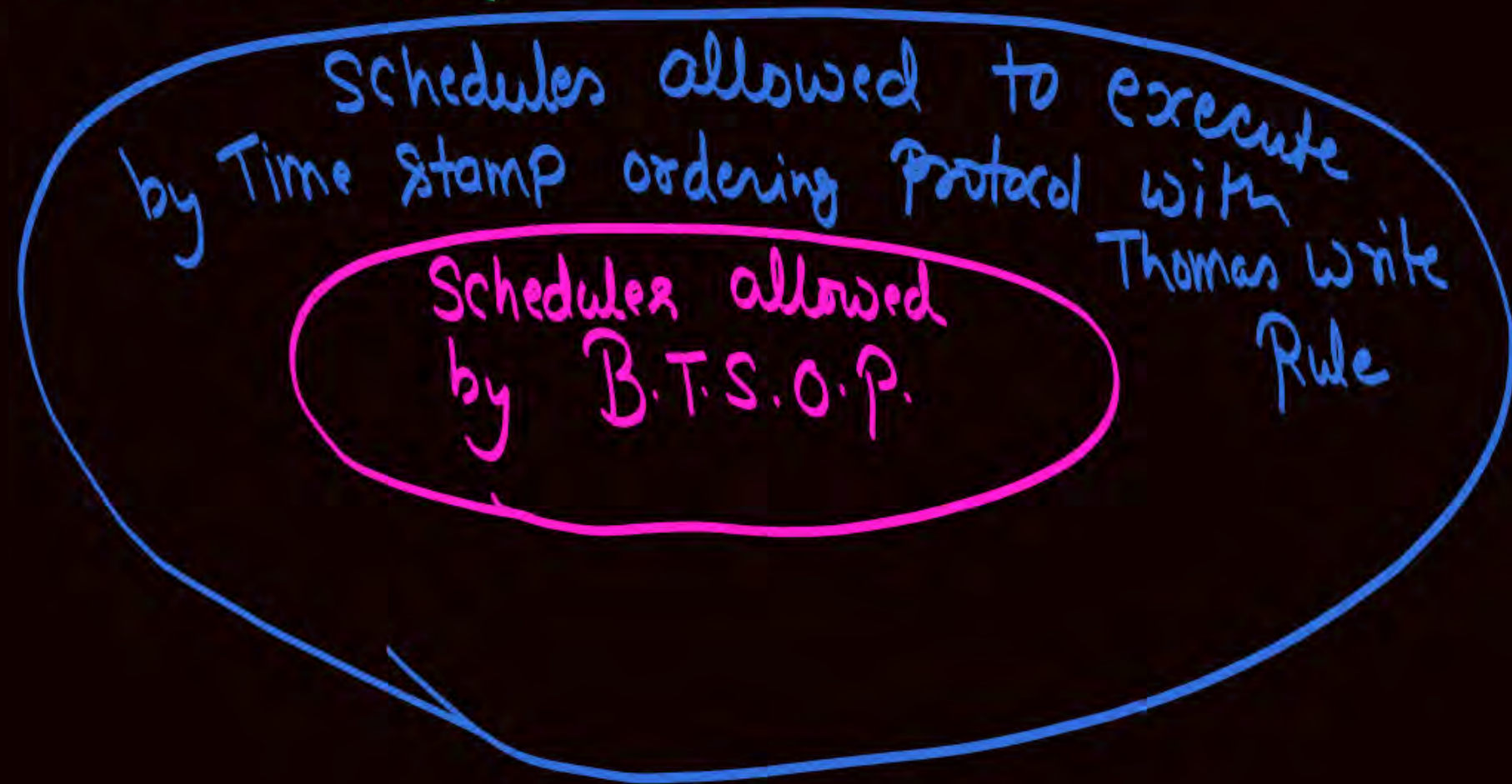
- ① If $WTS(A) > TS(T_1)$, then Rollback T_1
- ② Otherwise, T_1 is allowed to perform the Read(A) opⁿ
∴ T_1 will perform Read(A) opⁿ,
And set $RTS(A) = \text{Max}(RTS(A), TS(T_1))$

② Let T_1 issue Write(A) opⁿ:-

- ① If $RTS(A) > TS(T_1)$, then Rollback T_1 .
- (or) ② If $WTS(A) > TS(T_1)$, then ignore this w(A) opⁿ of transaction T_1
And Continue with the other operations of transaction T_1
- (or) ③ Otherwise, T_1 is allowed to perform Write(A) opⁿ.
∴ T_1 will perform the write(A) opⁿ
And Set $WTS(A) = TS(T_1)$

Notes :-

- ① Every schedule which is allowed by B.T.S.O.P. is allowed to execute using Time stamp ordering Protocol with Thomas write Rule, But Converse of the statement need not be true



Note:- ②

If A schedule is allowed to execute using Time Stamp ordering protocol with Thomas write Rule, then schedule produced by Time Stamp ordering protocol with Thomas write Rule will be view equivalent to the serial schedule based on time stamp ordering of the transactions.

Note :- ③ Both B.T.S.O.P & Time stamp ordering protocol with Thomas write rule are free from deadlock, but starvation is possible

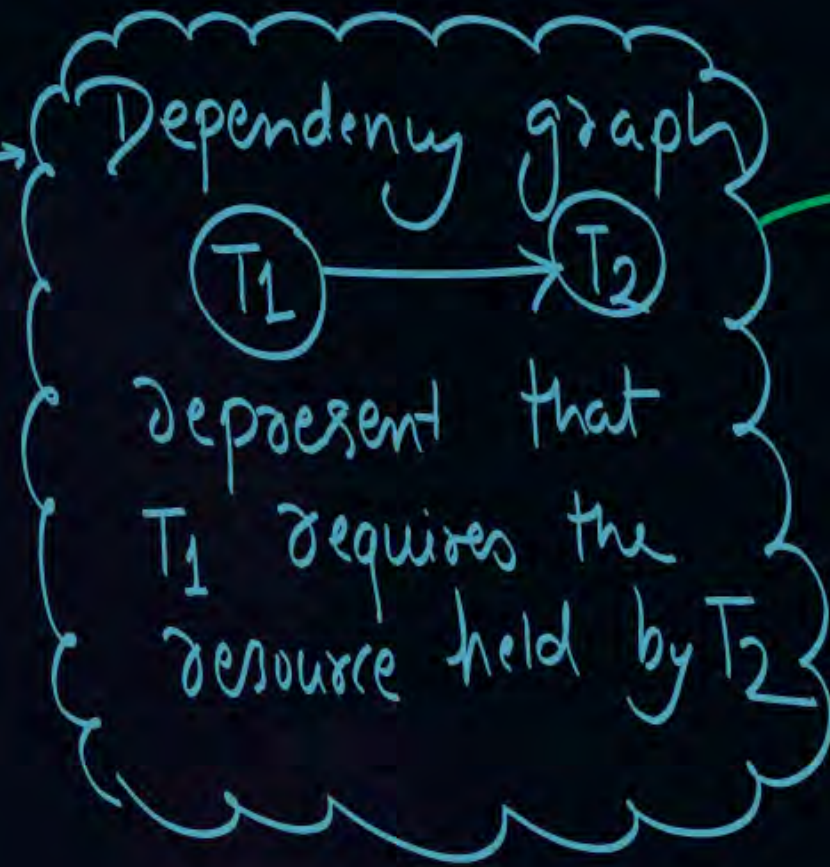
④ Schedules allowed by B.T.S.O.P and/or Time stamp ordering protocol with Thomas write rule may suffer from irrecoverability, cascading rollback and lost-update problem, To solve that problem we can implement Strict time stamp ordering protocols.



Topic : Deadlock prevention algorithms

→ We can use the Concept of time stamps assigned by DBMS to prevent the deadlock in lock based Concurrency Control Protocols.

If dependency graph is cyclic, then deadlock may be possible



If dependency graph is of the type

```
graph LR; T1((T1)) --> T2((T2)); T2 --> T3((T3)); T3 --> T1;
```

then it is cyclic dependency graph and Hence deadlock may be possible

If we can prevent from the formation of cycles in the dependency graph, then we can prevent from deadlock



Topic : Deadlock prevention algorithms

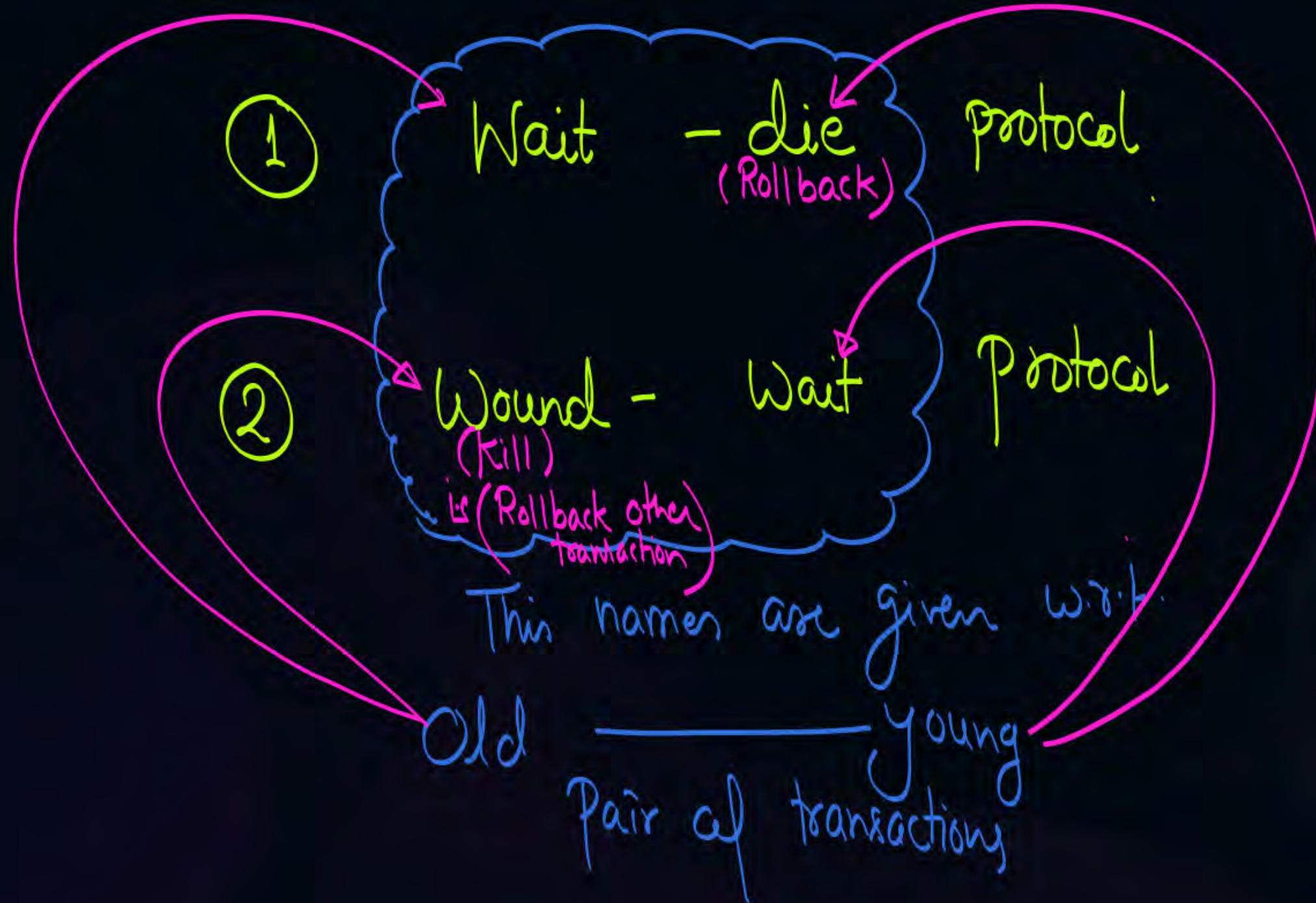
There are two algorithms (Protocols) that can be used to prevent the the formation of Cycles in the dependency graphs, and hence can be used to prevent the deadlock in lock based protocols.

Those algorithm are

- ① Wait - die protocol
- ② Wound - wait protocol.



Topic : Deadlock prevention algorithms



We will always Rollback young (new) transaction using Wait-die or Wound-wait Protocol



Topic : Wait-die algorithms

Old - young

If old transaction requires the resource held by young transaction then old transaction is allowed to wait

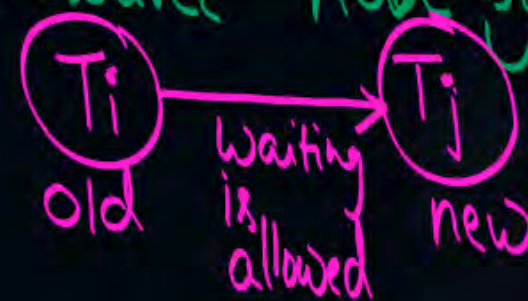
If young (new) transaction requires the resource held by old transaction, then young transaction will die. {i.e. young transaction} will Rollback

Let T_i & T_j are two transactions

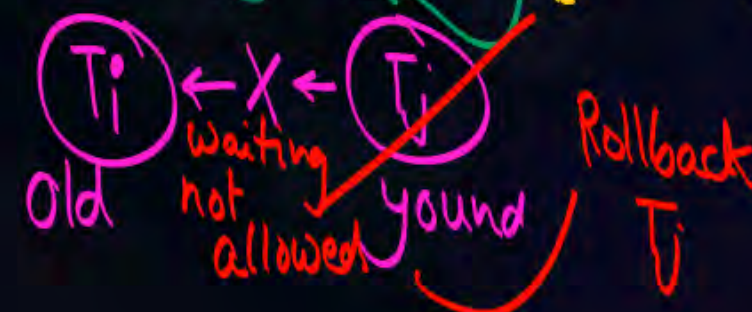
s.t. $TS(T_i) < TS(T_j)$

i.e. T_i is old & T_j is young

① If T_i (old) requires the resource held by T_j (new), then T_i is allowed to wait



② If T_j (young) requires the resource held by T_i then T_j is not allowed to wait and T_j Rollback



Waiting is allowed only in one direction
• formation of cycle is not possible



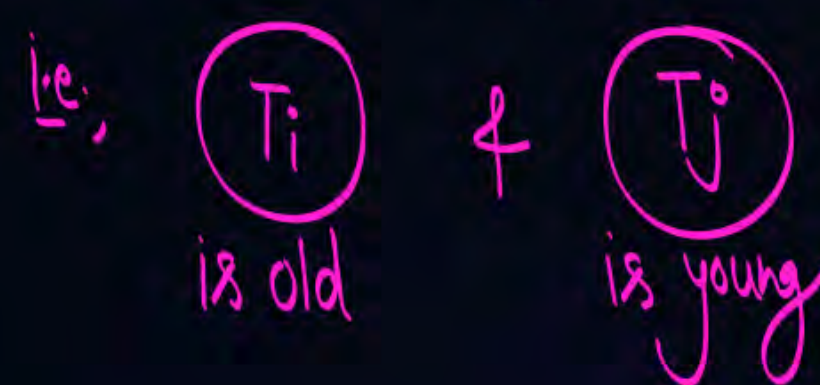
Topic : Wound-wait algorithms

old young

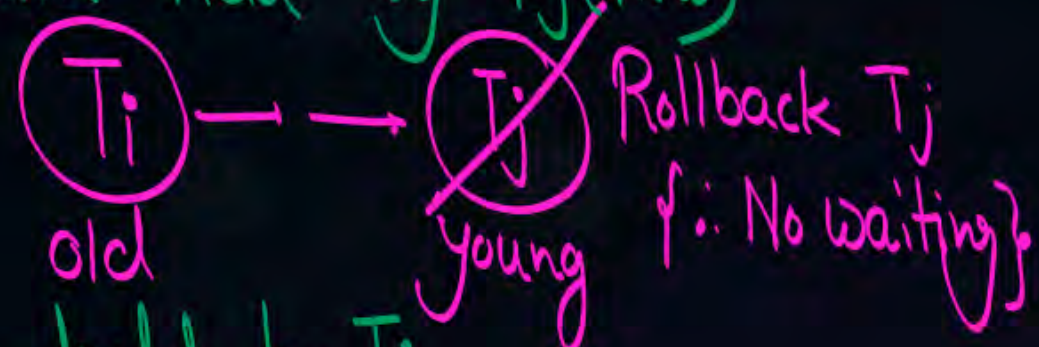
If old transaction requires the resource held by young transaction, then old transaction will wound (hurt/kill) young transaction
{i.e. young transaction will Rollback}

If young transaction requires the resource held by old transaction, then young transaction is allowed to wait

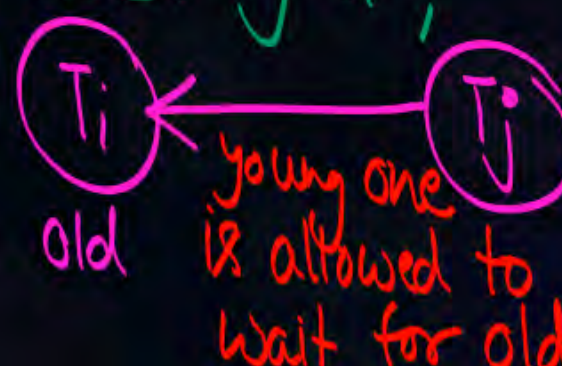
Let T_i & T_j are two transaction such that $TS(T_i) < TS(T_j)$



① If T_i (old) requires the resource held by T_j (New) then T_i will rollback T_j



② If T_j requires the resource held by T_i , then T_j is allowed to wait



* Using Wait-die & Wound-wait we can
Prevent from deadlock in lock based protocols, but
Starvation will still be possible.



2 mins Summary



✓
Topic

Time stamp ordering protocol with Thomas write rule

✓
Topic

Wait-die protocol

✓
Topic

Wound-wait protocol

THANK - YOU